

Rajalakshmi Engineering College

Name: SHREYA KS
Email: 240701506@rajalakshmi.edu.in
Roll no: 240701506
Phone: 9789293683
Branch: REC
Department: CSE - Section 10
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 37.5

Section 1 : COD

1. Problem Statement

The city library maintains a record of books available for lending. Each book is uniquely identified by its ISBN number, along with its title and author. The librarian wants to efficiently store and manage these records, ensuring books can be listed in the order they were added.

Your task is to implement a Library Management System using HashSet where:

The librarian adds books with ISBN, title, and author. The librarian can remove books by providing an ISBN. Finally, the librarian displays the available books in the order they were added.

Implement a class Library that will handle these operations. The main function should manage user input and interact with the Library class accordingly.

Input Format

The first line contains an integer n – the number of books to be added.

The next n lines contain three values: ISBN (integer), Title (string without spaces), and Author (string without spaces).

1. An integer employee_id
2. A string title
3. A string author name

The next line contains an integer m – the number of books to be removed.

The next m lines follow, each contains an ISBN number to remove.

Output Format

The output prints a list of books available in the library after performing all operations in the format:

"ISBN: <isbn>, Title: <title>, Author: <author>"

If no books remain, print: "No books available"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
1234 JavaCompleteGuide JohnDoe
5678 PythonBasics JaneDoe
9012 DataStructures AliceSmith
1
5679

Output: ISBN: 1234, Title: JavaCompleteGuide, Author: JohnDoe
ISBN: 9012, Title: DataStructures, Author: AliceSmith
ISBN: 5678, Title: PythonBasics, Author: JaneDoe

Answer

```
import java.util.*;
```

```
class Book {
    int isbn;
    String title;
    String author;

    Book(int isbn, String title, String author) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
    }

    @Override
    public int hashCode() {
        return Objects.hash(isbn);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Book b = (Book) obj;
        return isbn == b.isbn;
    }

    @Override
    public String toString() {
        return "ISBN: " + isbn + ", Title: " + title + ", Author: " + author;
    }
}
```

```
class Library {
    HashSet<Book> books = new HashSet<>();

    void addBook(int isbn, String title, String author) {
        books.add(new Book(isbn, title, author));
    }

    void removeBook(int isbn) {
        books.removeIf(b -> b.isbn == isbn);
    }

    void displayBooks() {
```

```

        if (books.isEmpty()) {
            System.out.println("No books available");
        } else {
            for (Book b : books) {
                System.out.println(b);
            }
        }
    }
}

```

```

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Library library = new Library();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int isbn = sc.nextInt();
            String title = sc.next();
            String author = sc.next();
            library.addBook(isbn, title, author);
        }
        int m = sc.nextInt();
        for (int i = 0; i < m; i++) {
            int isbn = sc.nextInt();
            library.removeBook(isbn);
        }
        library.displayBooks();
        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is working on a program that checks if one set of numbers is a subset of another. If Set B is a subset of Set A, the program should print "YES" followed by the sorted elements of Set B. If Set B is not a subset of Set A, the program should print "NO" followed by the average of all elements from both sets combined, rounded to two decimal places.

Implement a class Solution with the required method to perform the subset check using TreeSet in Java.

Input Format

The first line contains an integer n - the number of elements in Set A.

The second line contains n space-separated integers - the elements of Set A.

The third line contains an integer m - the number of elements in Set B.

The fourth line contains m space-separated integers - the elements of Set B.

Output Format

If Set B is a subset of Set A, print "YES" followed by the sorted values of Set B.

Otherwise, print "NO" followed by the average of all numbers in both sets (rounded to two decimal places).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2 3 4 5

3

2 3 5

Output: YES 2 3 5

Answer

```
import java.util.*;
```

```
class Solution {
```

```
    public static void checkSubset(TreeSet<Integer> setA, TreeSet<Integer> setB,  
    int totalCount, long totalSum) {
```

```
        if (setA.containsAll(setB)) {
```

```
            System.out.print("YES ");
```

```
            for (int x : setB) {
```

```
                System.out.print(x + " ");
```

```

    }
    } else {
        double avg = (double) totalSum / totalCount;
        System.out.printf("NO %.2f", avg);
    }
}
}

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        TreeSet<Integer> setA = new TreeSet<>();
        long sum = 0;
        for (int i = 0; i < n; i++) {
            int num = sc.nextInt();
            setA.add(num);
            sum += num;
        }
        int m = sc.nextInt();
        TreeSet<Integer> setB = new TreeSet<>();
        for (int i = 0; i < m; i++) {
            int num = sc.nextInt();
            setB.add(num);
            sum += num;
        }
        Solution.checkSubset(setA, setB, n + m, sum);
        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

A college professor wants to keep track of students who attend classes. Each student has a unique roll number and their attendance count increases every time they attend a class. The system should allow adding a student, marking their attendance, and displaying all students with their total attendance.

Your task is to implement a Java program using TreeSet to maintain students in sorted order of roll numbers and track their attendance count.

Operations:

A roll_no name Add a student with roll number and name (if not already added). M roll_no Mark attendance for the student with the given roll number (increase their count by 1). D Display all students in ascending order of roll number along with their attendance count.

Input Format

The first line contains an integer N - the number of students.

The next N lines contain one of the following commands:

A roll_no name

M roll_no

D

- A (Add) Adds a new student with a unique roll number and name.
- M (Mark) Increases attendance count for the given roll number.
- D (Display) Prints all students in ascending order of roll number.

Output Format

For D, output prints each student's roll number, name, and attendance count in ascending order of roll number.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

A 101 Alice

A 102 Bob

M 101

M 101

D

Output: 101 Alice 2
102 Bob 0

Answer

```
import java.util.*;

class Main {

    // Student class to store roll number, name, and attendance
    static class Student implements Comparable<Student> {
        int roll;
        String name;
        int attendance;

        Student(int roll, String name) {
            this.roll = roll;
            this.name = name;
            this.attendance = 0;
        }

        // Sorting based on roll number
        public int compareTo(Student other) {
            return this.roll - other.roll;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        TreeSet<Student> set = new TreeSet<>();

        while (N-- > 0) {
            String command = sc.next();

            if (command.equals("A")) {
                int roll = sc.nextInt();
                String name = sc.next();

                // Check if student already exists
                boolean exists = false;
                for (Student s : set) {
```



```

        if (s.roll == roll) {
            exists = true;
            break;
        }
    }

    if (!exists) {
        set.add(new Student(roll, name));
    }

} else if (command.equals("M")) {
    int roll = sc.nextInt();

    for (Student s : set) {
        if (s.roll == roll) {
            s.attendance++;
            break;
        }
    }

} else if (command.equals("D")) {
    for (Student s : set) {
        System.out.println(s.roll + " " + s.name + " " + s.attendance);
    }
}

}
}
sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Bob wants to develop a score-tracking application for a gaming tournament. Each player's score is stored in a HashMap with the player's name as the key and the score as the value.

Write a program to assist Bob that takes user input to enter player scores,

calculates the maximum score from the HashMap, and prints the player with the highest score.

Input Format

The input consists of strings representing player details in the format "playerName:score".

The input is terminated by entering "done".

Output Format

The output displays a string, representing the player's name who scored the maximum.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are given, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Alice:15

Bob:56

done

Output: Bob

Answer

```
import java.util.*;
```

```
class ScoreTracker {
```

```
    HashMap<String, Integer> scoreMap = new HashMap<>();
```

```
    // Returns false if any invalid input/format found
```

```
    public boolean processInput(String input) {
```

```
        // Input must contain exactly one colon and no other special characters
```

```
        if (!input.contains(":") || input.indexOf(':') != input.lastIndexOf(':')) {
```

```
            return false;
```

```
}  
    // Check for special characters excluding ':'  
    for (char c : input.toCharArray()) {  
        if (!Character.isLetterOrDigit(c) && c != ':') {  
            return false;  
        }  
    }  
}
```

```
String[] parts = input.split(":");  
if (parts.length != 2) return false;
```

```
String name = parts[0];  
String scoreStr = parts[1];
```

```
// Name validation  
if (name.length() == 0 || name.length() > 20) {  
    return false;  
}
```

```
// Score must be numeric  
for (char c : scoreStr.toCharArray()) {  
    if (!Character.isDigit(c)) {  
        System.out.println("Invalid input");  
        return false;  
    }  
}
```

```
int score = Integer.parseInt(scoreStr);  
if (score < 1 || score > 100) {  
    System.out.println("Invalid input");  
    return false;  
}
```

```
scoreMap.put(name, score);  
return true;  
}
```

```
// Find the player with the maximum score  
public String findTopPlayer() {  
    String topPlayer = "";  
    int maxScore = -1;
```

```

        for (Map.Entry<String, Integer> entry : scoreMap.entrySet()) {
            if (entry.getValue() > maxScore) {
                maxScore = entry.getValue();
                topPlayer = entry.getKey();
            }
        }

        return topPlayer;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ScoreTracker tracker = new ScoreTracker();
        boolean validInput = true;

        while (true) {
            String input = scanner.nextLine();

            if (input.toLowerCase().equals("done")) {
                break;
            }

            if (!tracker.processInput(input)) {
                validInput = false;
                break;
            }
        }

        if (validInput && !tracker.scoreMap.isEmpty()) {
            System.out.println(tracker.findTopPlayer());
        }

        scanner.close();
    }
}

```

Status : Partially correct

Marks : 7.5/10